

From Nand to Tetris

NPU Extension

Chapter 3: GEMM 控制器与卷积数据通路

GEMM Controller and Convolution Data Path

Assignment

Building a Modern Computer From First Principles

Introduction

本章作业包含控制器设计、RTL 实现与 golden 对比。

作业内容说明

本作业要完成什么

- 补全 `assignment/07/ReLU.v`：量化域 ReLU（负数清零）；
- 补全 `assignment/07/MaxPool.v`：2x2 最大池化；
- 补全 `assignment/07/ConvUnit.v`：3x3 卷积控制器 + 数据通路（接口与时序见下文）；
- 模板里只有模块声明，实现全部留空；参考实现在 `solution/07/`。
- 用 `make sim-07` 通过三个测试台：ReLU 256 项 + MaxPool 10 项 + ConvUnit 125 项，共 391 项检查。

ReLU 模块说明

- 量化域 ReLU：int32 累加结果重新量化到 int8 后，负数直接清零。
- 纯组合逻辑，零乘法；测试遍历 -128..127 全部输入。

端口	方向	位宽	说明
in	in	8（有符号）	输入激活
out	out	8（有符号）	输出 = max(in, 0)

MaxPool 模块说明

- stride=2 时输出宽高减半；只比较、不乘法，可跟在卷积输出后流式处理。
- 测试覆盖负数、平局与边界值（-128/127）。

端口	方向	位宽	说明
in0 / in1 / in2 / in3	in	8（有符号）	2x2 窗口的四个元素
out	out	8（有符号）	四个输入的最大值

ConvUnit 参数与端口

框架默认 C_IN=C_OUT=4（阵列 N=4）以控制 iverilog 仿真时间；把参数调成 8 即复用 8x8 阵列，逻辑不变。

参数	默认	说明
H / W	8 / 8	输入特征图高/宽
C_IN / C_OUT	4 / 4	输入/输出通道（须等于阵列 N）
K	3	卷积核尺寸
STRIDE / PAD	1 / 0	支持 stride=1/2、pad=0/1

A_W / W_W / P_W	8 / 8 / 32	激活/权重/累加位宽（有符号）
-----------------	------------	-----------------

注：阵列延迟 $LAT = 2 \times (N-1)$ 拍（本框架 $N=4$ 时为 6 拍）。

ConvUnit 端口

端口	方向	说明
clk / arst	时钟/复位	posedge；异步复位，高有效
wr_ifmap	in	装载特征图：为 1 时每拍写入一个像素
ifmap_in	in	一个像素的 C_IN 个通道（int8）
w_data	in	扁平权重 $W[oc][ic][ky][kx]$ ，start 时整体写入
start	in	特征图装载完成后启动一次推理
out_data	out	一拍一个输出像素的 C_OUT 个通道
out_valid / done	out	输出有效 / 整幅图推理完成（脉冲一拍）

计算语义

- $out[oy][ox][oc] = \text{relu_clamp}(\text{sum}_{\{ky,kx,ic\}} \text{ifmap}[oy*STRIDE+ky-PAD][ox*STRIDE+kx-PAD][ic] * W[oc][ic][ky][kx])$
- 窗口越界按 0 补齐； $\text{relu_clamp}(x) = (x < 0) ? 0 : (x > 127 ? 127 : x)$ 。
- $3 \times 3 \times C_IN$ 的 im2col 行长度 $9 \times C_IN$ 超过阵列宽度，按 (ky,kx) 拆成 9 个 tap；
- 每个 tap 是一次 $C_IN \times C_OUT$ GEMM，外部 int32 累加器跨 tap 累加，得到一个输出像素的所有输出通道。

时序与状态机

- FSM：IDLE -> LOAD_W -> RUN -> DRAIN -> EMIT -> DONE。
- IDLE：wr_ifmap=1 期间逐拍装载特征图；start 后进入 LOAD_W。
- LOAD_W：拉高 w_load 两拍，把第 tap 个权重矩阵写入阵列。
- RUN：逐拍送 OPIX 个输入向量并收集输出；控制器用 warmup=LAT+1 跳过阵列填充期的无效输出，保证累加索引对齐。
- 每个 tap 送完后补 FLUSH 个 0 冲掉流水，否则上一 tap 残留在阵列里的激活会污染下一 tap。
- DRAIN：pending==0（本 tap 全部结果收完、流水已清）后切下一个 tap。
- EMIT：逐拍输出 relu_clamp 后的像素（out_valid=1）；DONE：done 脉冲一拍，回 IDLE。

验证方法

- golden 内嵌在 testbench 中（int8 整数运算，避免浮点误差），与 RTL 逐元素比对。

- ConvUnit_tb 四组配置并行：stride=1/pad=0（36 像素）、stride=1/pad=1（64 像素）、stride=2/pad=0（9 像素）、stride=2/pad=1（16 像素），共 125 项检查。
- 运行：`make sim-07`（学生模式，只跑 NPU 三题）；`make sim-07 RTLDIR=solution`（答案回归，含 posedge Hack 已知库）。
- 全连接层 = ConvUnit 配
K=1：没有滑动窗口，直接逐拍喂输入向量，复用同一套控制器与阵列。
- 常见错误：忘记 signed 导致负数乘法错；tap 之间不冲流水；累加器不复位；warmup 错一拍导致输出整体平移。

Questions

Q1. 画出卷积控制器的 FSM，标注每个状态做什么、什么时候跳转。

Hint: 至少包含 IDLE/LOAD_W/RUN/DRAIN/DONE。

Q2. 输入 28x28x8、3x3 卷积、输出 8 通道，im2col 后矩阵的维度是多少？每个矩阵元素来自哪里？

Hint: $K = 8 \times 3 \times 3 = 72$ ，输出像素数 = 28×28 。

Q3. 计算 28x28x8 输入 SRAM 和输出 SRAM 的最小容量（int8，含 padding 考虑）。

Hint: $28 \times 28 \times 8 = 6272$ 字节；padding 后边界像素需要额外存储。

Q4. 解释 line buffer 如何降低带宽：与 im2col 相比各有什么优缺点？

Hint: 重复读取次数 vs 硬件复杂度。

Q5. 设计 golden 生成策略：如何保证 Python 参考实现与 RTL 数值一致？

Hint: 用整数运算、避免浮点；逐层导出。

What to Submit

- 控制器、激活/池化、FC 的 RTL 源码。
- golden 数据与 testbench。
- `make sim-07` 的 PASS 输出。
- Q1-Q5 书面回答。

Rubric

Item	Points
FSM 设计正确	20
卷积与 golden 一致	30
ReLU/MaxPool 正确	15
FC 复用阵列正确	15

书面推导与带宽分析	20
-----------	----

Total: 100 points